

과제 2 — AI 활용 산출물 및 과정

무엇을 만들었나

"투자 아이디어 → 코드 → 백테스트" **파이프라인(Research-to-Backtest)**. 사람이 세운 투자 논지(과제 1의 네이버 "실적 확인형" 같은)를 ① DART·KRX 데이터 수집·정규화 ② 전략 규칙(DSL) ③ Point-in-Time 백테스트 ④ 근거·리포트 자동화로 잇는 시스템이다. 펀드매니저의 아이디어를 Python 쿼리 모델로 구현·검증하는 업무를 그대로 도구화한 것이다.

- **산출물**: GitHub 리포지토리(첨부 링크) — 파이썬 45k 라인, 테스트 837건, 실데이터 end-to-end 실행, 실행 가이드 README.
- **실행 예**: 네이버 논지를 전략 규칙으로 변환해 2016~2025 백테스트까지 CLI/화면으로 관통.

어떤 도구·프롬프트로

두 층위로 AI를 썼다.

(1) 만들 때 — **Claude Code 멀티에이전트 오케스트레이션**. 메인 세션(설계·명세·병합·검증)이 구현을 **워크트리 격리 병렬 에이전트**에 위임했다. 여기서 **프롬프트 = 명세 파일**이다 — docs/specs/*.md 에 파일 소유권과 인터페이스 계약을 먼저 고정해 병합 충돌을 설계 단계에서 차단했다. 4개 웨이브·12개 병렬 트랙에서 병합 충돌은 1건이었다.

명세 프롬프트 예시(발췌): "너는 T1 구현 에이전트다. §1 파일 소유권 밖 수정 금지. build_financial_datasets를 조립해 CLI를 붙이고, 실데이터 스모크로 136 metrics 재산출을 확인하라. 명세와 실측이 다르면 우회하지 말고 사유와 함께 보고하라."

(2) 시스템 안에서 — **Claude(Haiku)**. 파이프라인 내부에서 LLM은 **후보 정리·초안만** 말한다(분석 후보·가설 후보·전략 DSL 초안·결과 설명). 모델은 configs/llm.yaml 에 고정, 프롬프트는 */prompts/*_v1.txt 로 버전 관리, 모든 호출은 ai_usage_log.jsonl 에 기록된다.

어떻게 풀었나 (과정과 판단)

- **역할 분리를 코드로 강제**: AI는 후보·초안, 사람은 관점·가설·승인·해석. 승인 게이트 (12-상태 머신)가 미승인 단계 실행을 예외로 막는다 — "AI가 판단한 것처럼 보이는" 구조를 회피했다(과제 1의 "본인의 시각" 요건과 직결).
- **데이터 신뢰성 우선**: 모든 재무값에 공시 접수 다음 거래일(available_from)을 부여하고 as-of 결합만 허용해 미래 정보 유입을 차단(Point-in-Time). API 수치와 XBRL 원본을 교차검증(연간 100% 일치)한 뒤에만 AI에 근거로 넘긴다.
- **AI 산출물을 믿지 않고 검증**: LLM이 인용한 근거 ID가 실제 데이터에 없으면 자동 재요청, 전략 초안은 컴파일러로 재검증, 결과는 사람이 해석·판정한다.
- **한계도 정직하게**: 하이퍼클로바X 단독 손익처럼 공개되지 않은 값은 추정하지 않고, 백테스트가 검증하는 것은 논지의 "재무 확인" 대리 명제까지임을 명시했다.

검증 가능성

- 개발 과정: git 커밋 이력·Co-Authored-By 트레일러·docs/PROGRESS.md (웨이브별 스냅샷).
- 시스템 내 AI 사용: outputs/{run_id}/ai_usage_log.jsonl + 프롬프트 버전 파일.
- 동작: make check (테스트·타입·린트) 및 실데이터 실행 로그.

우대사항 연결 — 충족 범위와 한계 (정직하게)

- **충족**: 공시·거래소 **데이터 API**(OpenDART·KRX) 연동으로 금융 데이터 크롤링·수집·가공, 백테스트(**시뮬레이션**) 플랫폼 산출물, AI로 Python 기술 허들을 스스로 넘어 구현·검증한 사례. 데이터 수집 계층은 어댑터(MarketDataSource 인터페이스)로 추상화해 다른 증권사 데이터 소스로 교체 가능하도록 설계했다.
- **미충족(솔직히)**: 실제 증권사(브로커리지) 주문 API를 통한 시스템 트레이딩(라이브 주문 집행)은 이번 산출물에 포함하지 않았다. 현재 플랫폼은 신호 생성 → 체결 시뮬레이션 → 성과·강건성 검증까지를 담당하며, 주문 집행 계층은 미구현이다.
- **다음 단계**: 토스증권 API 키는 발급·등록을 마쳤다(.env.example). 백테스트에서 검증된 전략을 페이퍼트레이딩 → 실주문으로 연결하는 집행 어댑터를 추가하는 것이 자연스러운 확장 방향이며, 데이터 소스와 동일한 어댑터 패턴으로 붙일 수 있다.

"시스템 트레이딩"을 데이터 수집(주요업무)과 실주문 집행(우대사항)으로 구분해 표기했다 — 이번 제출물은 전자를 실연하고 후자는 로드맵으로 남긴다.